这是一份为你量身定制的 Founder OS V1.0 (MVP) 产品需求文档 (PRD)。

这份文档不仅是写给研发团队的"工程蓝图"，更是你向外融资、对内对齐产品心智的"商业白皮书"。它彻底摒弃了传统 AI 聊天机器人的设计套路，将我们探讨的"管理学法则"与"IronClaw 底层安全技术"进行了完美的工程化映射。

Founder OS V1.0 产品需求文档 (PRD)

文档状态: V1.0 (最小可行性产品 MVP)

产品定位: 专为超级个体/初创 CEO 打造的"数字企业 ERP 与虚拟高管调度中心"。

核心信条: 软件设计即组织设计。消灭聊天框，让创始人回归"决策者"角色，而非"提示词打字员"。

一、产品愿景与设计哲学 (Vision & Philosophy)

1. 核心痛点

- 极度的上下文切换：创始人每天在几十个 SaaS 工具、数据报表和群聊中切换，缺乏统一的指令下达出口。
- AI 系统的"大企业病"：市面的多 Agent 框架本质是"大模型自由群聊"，导致极高的沟通摩擦（协调成本 > 产出）、上下文污染和幻觉。
- 安全与越权恐惧：不敢把公司核心业务 API 和资金权限交给大模型自由决断，害怕 AI "发疯"造成不可挽回的灾难。

2. 管理学映射的设计原则

- 部门边界极简 (Workspace)：MVP 版本只有 1 个大盘和 1 个员工（幕僚长）。拒绝"差生文具多"，避免跨部门协调成本。
- 培训优于招人 (Skill > Agent)：遇到新需求，优先为幕僚长加载新 Skill（SOP 注入），认知爆表前绝不新建 Agent。
- 保护认知负荷 (Context Window)：采用"按需知晓 (Need to know)"原则，任务完成立刻清空上下文，绝不全量加载知识库。
- 按错误成本划分权限 (Cron vs Approval)：低风险操作走后台静默 Cron；高风险操作强制拦截，生成「审批卡片」让人类在环。

二、系统三层架构选型 (Architecture Stack)

采用 "胖底座，强中间件，薄应用" 的三层架构：

层级	技术选型	核心职责	管理学映射
展现层 (UI/UX)	Next.js + TailwindCSS	CEO 指挥中心（全局指令、审批收件箱、大盘）。绝对禁止提供对话气泡界面。	老板的办公桌与可视化看板
中间件 (Middleware)	Node.js (NestJS) / Python	拦截高危操作挂起、管理记忆垃圾回收 (GC)、强制校验交接格式。	制度合规部、流程审批枢纽
底座层 (The Engine)	IronClaw (Rust)	WASM沙盒物理隔离、加密密钥金库(Vault)、底层定时心跳唤醒。	物理安保、基础设施、员工脑力

三、核心功能模块与 UI 规约 (Core Modules)

模块 1：全局指挥舱 (Cmd+K Command Bar) [优先级：P0]

- 描述：创始人唯一的交互入口，彻底替代冗长的对话框，类似 Mac Spotlight。
- 用户故事：作为 CEO，我需要随时下达模糊指令并立刻关掉窗口去忙别的，系统应在后台自动拆解执行。
- 功能需求：
  - 在任何界面按 Cmd+K 唤出居中悬浮输入框。
  - 支持自然语言输入（例："下周产品发布，起草一封给早期用户的预热邮件，并在推特上发条 Teaser"）。
  - 敲击 Enter 后，输入框立刻消失，系统右上角弹出静默 Toast 提示："任务已指派给幕僚长..."。

模块 2：决策收件箱 (Approval Inbox) [优先级：P0] -- 核心商业壁垒

- 描述：Founder OS 的主界面（视觉 C 位），拦截所有高风险 AI 操作，强制人类在环审批。
- 触发条件：调用带有 High-Risk 标签的 Skill（如：发邮件、退款、改代码）时，中间件将底层的 WASM 进程强行挂起（Suspend）。
- 审批卡片 (Approval Card) 要素：
  - 意图摘要：AI 为什么要做这件事（如：监控到竞品降价，故生成应对邮件）。
  - 执行预览：即将发出的邮件正文 / 预估消耗金额 / API Payload 明文对比。
  - 三大操作按钮：
  - Approve (批准)：中间件向 IronClaw 发送放行指令，注入 Vault 密钥，执行最后一步。
  - Reject (拒绝)：任务终止，要求填写一行拒绝原因（如："语气太凶了"），打回重写。
  - Modify (修改)：允许创始人直接在卡片上修改邮件措辞或参数，保存并放行。

模块 3：自动化大盘 (Silent Dashboard) [优先级：P1]

- 描述：反映低风险、重复性 Cron 任务成果的只读数据看板。
- 功能需求：
  - 无交互纯展示：好员工应该自己把报表画好放在老板桌上。没有任何输入框。
  - 数据流转：由 IronClaw 底层的 Cron 定时触发 WASM Skill 抓取数据，提炼为 JSON 写入中间件数据库，前端实时拉取渲染。
  - 组件示例：昨日 MRR 增量、竞品动态红绿灯、高优待处理客诉数量。

模块 4：渐进式记忆与 SOP 系统 (Progressive Memory) [优先级：P1]

- L1 全局价值观 (Global_Memory.md)：限 300 字，永久注入所有指令。定义公司使命和创始人沟通红线（例："对外口吻专业克制，绝不说废话"）。
- L3 动态 SOP 挂载：允许为 Skill 上传 Markdown 格式的 SOP。平时不占用 Token，只有触发对应意图时才瞬间加载（按需知晓原则）。
- L4 强制垃圾回收 (Garbage Collection)：任务在 Inbox 被 Approve 并执行完毕后，触发中间件逻辑：生成一句话核心结论归档，立刻强制清空本次会话的上下文，防止系统越用越卡。

四、MVP 核心业务闭环流转说明 (Key Workflows)

为了"吃自己的狗粮"并验证商业逻辑，V1.0 必须跑通以下两条极简的业务线：

用例 A：绿区静默自动化 (映射"规则明确的 Cron")

- 触发：IronClaw 底层 Cron 每天早上 8:00 唤醒"幕僚长"。
- 执行：调用被赋予的 Fetch_SaaS_Metrics Skill（只读属性，低风险）。
- 处理：抓取 Stripe 报表，提取 3 个核心指标。
- 输出：直接写入 Dashboard 数据库。全过程零老板干预，CEO 9 点打开 OS 界面即看。

用例 B：红区任务审批流 (映射"高风险决策交接")

- 指令：CEO 通过 Cmd+K 输入："给最近流失的用户 John 发一封邮件，提供 8 折挽留"。
- 规划：幕僚长理解意图，加载《挽回客户 SOP》，调用 Send_Email Skill。
- 拦截：中间件发现该 Skill 属于红区动作，立刻挂起底层执行进程。
- 展示：前端 Approval Inbox 生成审批卡片，展示生成的打折邮件草稿。
- 决策：CEO 点击卡片上的 Modify，把 8 折改成了 9 折，然后点击 Approve。
- 执行：中间件携带修改后的参数，向 IronClaw 放行。底层从加密金库提取 SendGrid Key 正式发信。任务状态变更为"Done"，触发垃圾回收清空上下文。

五、数据流与接口规约 (Handoff Contract)

为了彻底消灭"我以为你知道"的协作灾难，所有内部任务派发必须符合强类型 JSON Schema：

```
// Agent调取复杂Skill时的强制Payload格式
{
  "task_id": "REQ-1024",
  "from": "Chief_of_Staff",
  "to": "Skill_Stripe_Refund",
  "context": "大客户John提出退款要求，历史无违规记录。",
  "goal": "执行全额退款 $500",
  "constraints": [
    "若检测到订单已超过 30 天，必须抛出 Error，严禁静默失败"
  ]
}
```

注：若底层大模型输出的 JSON 不符合此 Schema，中间件将直接在后台发起重试，绝不将乱码或 Debug 代码推给前台老板审批。

六、研发排期与里程碑 (3-Sprint Roadmap)

预估周期：3 - 4 周（单人全栈 / 2人敏捷小队）

Week 1: 底层基建

- 核心任务：
  - 1. 私有化部署 IronClaw Rust 引擎。

- 2. 建立加密 Vault 并放入测试密钥。
- 3. 编写 2 个 WASM 测试技能（1 绿 1 红）。

- 验收标准：通过 CLI 能成功调起 IronClaw WASM 沙盒，且测试动作能被底层安全策略捕捉。

Week 2: 核心中间件

- 核心任务：
  - 1. 开发 Node.js/Python API 网关。
  - 2. 开发审批拦截器：拦截红区动作并存入 DB (Status: Pending)。
  - 3. 开发认知负荷清道夫 (GC) 逻辑。
- 验收标准：发起高危请求时，IronClaw 暂停执行，DB 成功生成结构化待办记录。

Week 3: CEO 面板

- 核心任务：
  - 1. Next.js 搭建 Cmd+K 组件。
  - 2. 开发 Approval Inbox UI 及卡片。
  - 3. 联调前端 Approve 动作与底层放行逻辑。
- 验收标准：前端点击 Approve 后，底层的 IronClaw 恢复执行并真实发出测试邮件。

Week 4: 测试闭环

- 核心任务：
  - 1. 完善 Dashboard 数据渲染。
  - 2. 创始人"吃狗粮"内测真实业务场景。
- 验收标准：跑通"早上自动出报表"与"人工审批发邮件"两个完整 User Journey。

七、成功指标与安全红线 (Success Metrics & NFR)

- 零 Token 泄漏 (Security First)：在任何前端代码、中间件数据库、乃至大模型的 Prompt 日志中，绝对不能出现第三方 SaaS 的 API Key 明文。必须 100% 依赖 Vault 的最后一公里注入。
- 审批决策时长 (Time-to-Decision)：创始人在 Inbox 中处理一张审批卡片的平均时间目标为 < 15秒。（说明 AI 总结的 Diff 足够清晰，无需追问即可拍板）。
- 静默执行率 (Silent Execution Rate)：由 Cron 触发且无需老板干预成功跑完的任务比例目标为 > 60%。